

Software Engineering



Dr. Fatma Eskander
Math.department, Faculty of Science, Mansoura
University

Lecture 3

Software Engineering Requirements



The Software Requirements & Requirements Engineering

What are requirements? Why are they important?

Defining Requirements Engineering

Criteria of a good Requirement

Requirements completeness & consistency

System stakeholders

Requirements classifications

What Are Requirements?

“ .. A requirement is a necessary attribute in a system, a statement that identifies a capability, characteristic, or quality factor of a system for it to have value and utility to a customer or user.”

“ .. A statement that identifies a product or process operational, functional, or design characteristic or constraint, which is unambiguous, testable or measurable, and necessary for product or process acceptability (by consumers or internal quality assurance guidelines).” *~IEEE-STD-1220-1998 (IEEE 1998)*

What Are Requirements?

What is a Software Requirement?

A **description** of what a system must do and the **constraints** under which it must operate.

Two Main Types:

Functional Requirements

"What the system must DO."

Example: "The system shall allow users to reset their password."

Non-Functional Requirements

"How well the system must perform its functions."

Examples: Performance, Security, Usability, Reliability.

Why Are Requirements Important?

“ .. Requirements are important because they provide the basis for all the development work that follows. Once the requirements are set, developers initiate the other technical work: system design, development, testing, implementation, and operation.”

“Without requirements or design, programming is the art of adding bugs to an empty text file.”

- *Louis Srygley*

اخطاء Buges

Why Are Requirements Important?

The Bedrock of Success

Define Scope: Set clear boundaries for the project.

Guide Development: Provide a blueprint for designers and developers.

Manage Expectations: Ensure alignment تنسيق between clients and the development team.

Reduce Costs: Fixing a requirements error early is far cheaper than fixing it after coding.

Quote: "The hardest single part of building a software system is deciding precisely what to build." - Fred Brooks

Requirements Engineering

The process of establishing the **services** that the customer requires from a system and the **constraints** under which it operates and is developed.

The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

Requirements engineering

Definition: A Structured Process for Getting Requirements Right. It is the systematic process of **discovering, analyzing, documenting, and maintaining** requirements.

Key Phases:

Elicitation → Gathering needs

Analysis → Understanding & prioritizing

Specification → Documenting clearly

Validation → Checking for accuracy

Management → Handling changes

Why Requirements Engineering?

“ .. It's not just a simple matter of writing down what the customer says (s)he wants.”

- A fundamental problem in business is that **requirements are inherently dynamic**; they will change over time as our understanding of the problem we are trying to solve changes.
- The importance of good requirements and the underlying dynamic nature of the process mean that we must be **(1) as accurate as possible**, and yet be **(2) flexible** (*i.e., we have a process for developing requirements & accommodating changed requirements as we clarify the real requirements of customers*).

Stated vs. real requirements

1. Stated Requirements

What are they? These are the requirements that a **user or customer explicitly provides** at the beginning of a project. They are based on their initial, and often limited, understanding of their own needs.

Characteristics:

They are often **vague غامض**, **superficial سطحية**, or **focus on solutions rather than the root problem** (e.g., "I want a bigger blue button").

They can be **incomplete** because the user may forget important details.

They can be **inaccurate or open to interpretation**. غير دقيقة أو قابلة للتفسير.

Real-World Example:

The client says: "We need a system that lets employees log in with a password." (This is the *stated* requirement).

Stated vs. real requirements

2. Real Requirements

What are they? These are the **verified, underlying needs** that the system must fulfill اكمل انجز . They are revealed through deep analysis by skilled Requirements Analysts.

Characteristics:

They are **precise and specific**, describing the core problem, not just a symptom.

They are **measurable and testable**.

They are often **discovered** by asking the right questions and observing the user's workflow.

Stated vs. real requirements

Real-World Example (building on the previous one):

After analysis, the **Real Requirement** is uncovered: "A secure system that verifies user identity (potentially using strong passwords or two-factor authentication) to prevent unauthorized access to sensitive data."

Notice how "secure" and "prevent unauthorized access" are the *real needs*, while "a password" was just one possible *solution*.

The Critical Role of the Requirements Analyst (RA)

This is the **core idea** and the "Why" behind Requirements Engineering:
The analyst's job is NOT just to collect and write down the stated requirements.

Their primary mission (role) is to:

Investigate and Dig Deeper التحقيق والتعمق : Like a detective, they ask "Why?" and "What happens if?" to understand the underlying need.

Validate: Ensure that the stated requirements will actually solve the right problem.

Identify Omitted Requirements تحديد المتطلبات المهملة : This is a key task. Often, the most critical system requirements are the ones the customer *didn't state* because they seem "obvious" or because the customer isn't aware of the technical possibilities.

In our example: The analyst is the one who might discover that the real requirements also demand:

- A password recovery system for forgotten passwords.

- A password policy (length, complexity).

- Different authorization levels for managers.

The gap between "Stated" and "Real" requirements is a primary reason why software projects fail. The success of a system depends entirely on the development team's ability to bridge this gap.

Summary

Aspect	Stated Requirements	Real Requirements
Source	Customer / User	Uncovered by the Requirements Analyst
Nature	Superficial, often suggested solutions	Fundamental, describing the core need and root problem
Accuracy	Can be vague or incomplete	Precise, complete, and measurable
Purpose	The starting point for the conversation	The true foundation for a successful system

Criteria of a Good Requirement

A Good Requirement Should Be **SMART**

Specific - Clear غير غامض , unambiguous, and concise.

Measurable - Can be tested and verified.

Achievable - Technically and financially feasible. واقعي من الناحية التقنية والزمنية.

Relevant - Aligns with business goals and project scope. يخدم أهداف المشروع.

Traceable - Can be linked to its origin and through testing. يمكن تنفيذه في الإطار الزمني المحدد.

Completeness & Consistency

Ensuring High-Quality Requirements

Completenessالاكتمال

Have we captured **everything** the system needs to do?

Are all user stories, functions, and constraints defined?

Goal: No missing pieces.

يجب أن تغطي المتطلبات جميع الوظائف والقيود المطلوبة من النظام، دون نقص.

Consistencyالاتساق

Do the requirements **contradict** each other?

Example: "The system must be highly secure (using complex encryption)" vs. "The system must respond in under 0.1 seconds on low-end hardware."

Goal: No conflicting statements.

يجب ألا تتعارض المتطلبات مع بعضها البعض.

Key Takeaways

Requirements define **WHAT** and **HOW WELL**.

Requirements Engineering is the disciplined process to get them right.

Good requirements are **SMART**.

Completeness and **Consistency** are critical to avoid rework and failure.

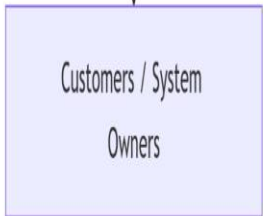
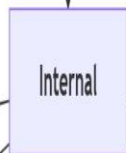
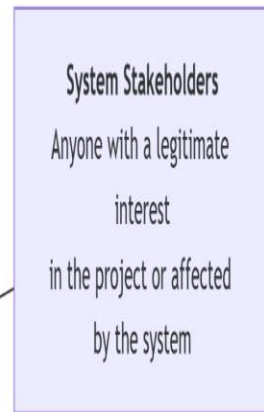
System stakeholders

Definition: A stakeholder is any **person, group, or organization** with a legal interest in a project or who is affected/influenced by the proposed system.

التعريف: صاحب المصلحة هو أي فرد أو مجموعة أو منظمة لها مصلحة شرعية في المشروع أو التي تتأثر/تتأثر بالنظام المقترح.

Why this matters: Identifying all stakeholders early is very important for understanding the **full scope of requirements** and ensuring the project's success. Without considering a key stakeholder, you might miss critical needs, leading to project failure

لماذا هذا مهم: يعد تحديد جميع أصحاب المصلحة في مرحلة مبكرة أمرًا بالغ الأهمية لفهم النطاق الكامل للمتطلبات وضمان نجاح المشروع. دون أخذ مصلحة طرف مهم في الاعتبار، قد تفوتك احتياجات حرجية، مما يؤدي إلى فشل المشروع.



System stakeholders Examples

Customers/Investors (Paying for the system)

End Users (People who directly use the system)

Project Teams (Developers, Testers, Analysts)

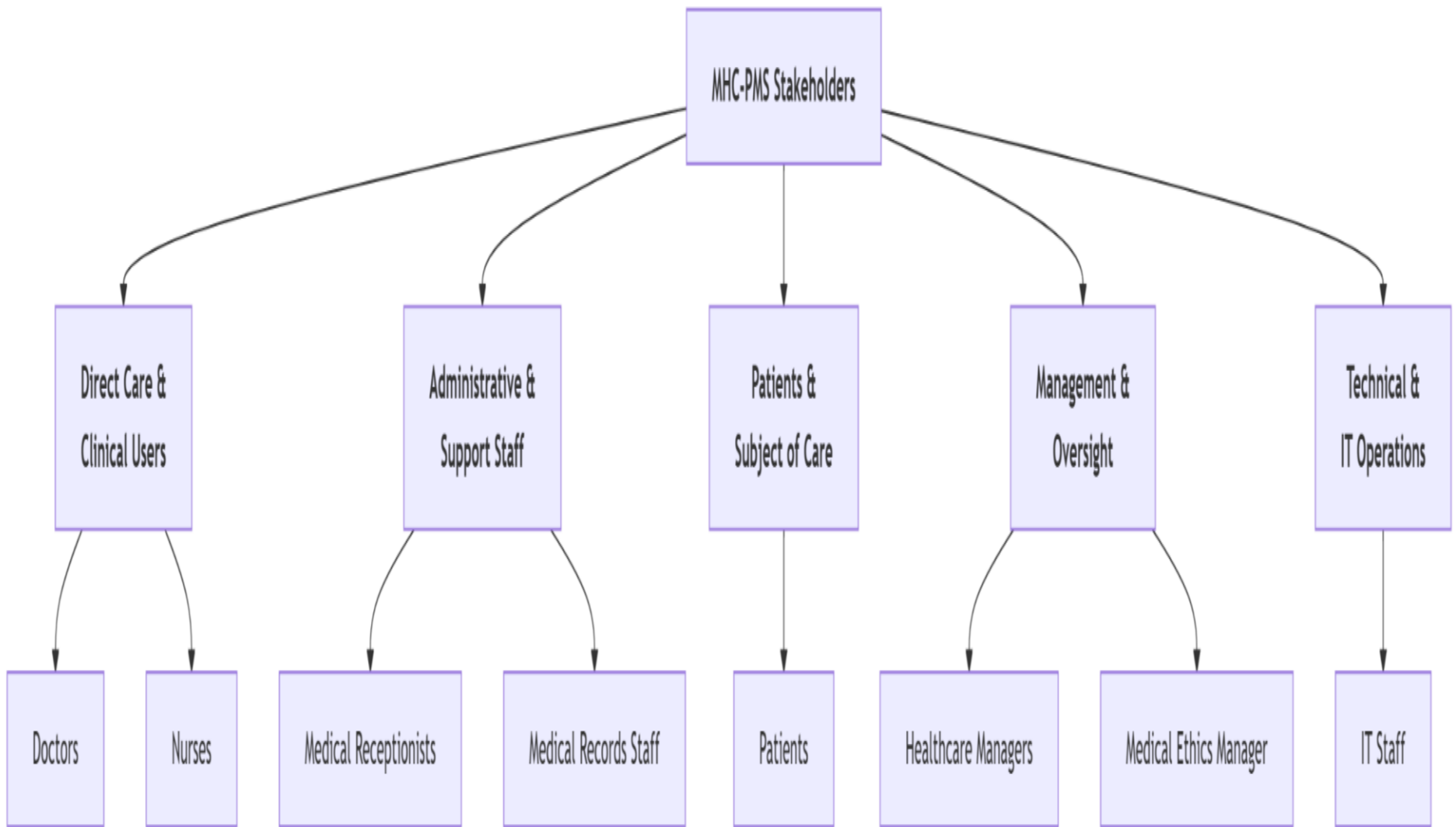
Regulatory Authorities (Legal, Compliance)

Maintenance Staff (Keep the system running)

Stakeholders of a Mental Health Care System (MHC-PMS) (example)

The diagram below visually categorizes the key stakeholders involved in the Mental Health Care Patient Management System (MHC-PMS), showing the diverse groups with an interest in the system's success.

Why this matters: Each group has unique needs from the system. For example, **Doctors** need quick access to patient histories, **IT Staff** need the system to be secure and reliable, and the **Medical Ethics Manager** needs to ensure patient data is handled confidentially and ethically. A successful system must meet the requirements of all these stakeholders.



Views of Requirements Classifications

Type of Requirement	Description & Focus	Example for a Bank Mobile App
Business Requirements <i>(Why we are building the system)</i>	The high-level goals and reasons for the project. They define the value for the organization.	"The app must increase customer retention by 15% by making banking more convenient."
User Requirements <i>(What the user should be able to do)</i>	The tasks a user must be able to perform with the system, described from a user's perspective.	"As a user, I want to transfer money to another person using just their phone number so that it is fast and easy."
Functional Requirements <i>(How the system will do it)</i>	The specific behaviors and functions the system must perform to satisfy the user requirements.	1. The system shall allow the user to enter a recipient's phone number. 2. The system shall validate the phone number is registered. 3. The system shall confirm the transfer with a pop-up before executing.
Non-Functional Requirements <i>(How well the system performs)</i>	The quality attributes and constraints under which the system must operate.	Security: Login must use two-factor authentication. Performance: 95% of transactions must process in under 2 seconds. Usability: The app must be navigable using screen-reading software.
Domain Requirements <i>(Rules from the field)</i>	Constraints derived from the application domain's laws,	"The system must record and encrypt all transactions as per the National Banking Regulations, Article

these classifications help ensure that all aspects of the system are considered—from the **strategic business goals** down to the **technical and regulatory details**. They work together to define a complete picture of what the system must be.

تساعد هذه التصنيفات على ضمان أخذ جميع جوانب النظام في عين الاعتبار—بدءاً من أهداف العمل الاستراتيجية وصولاً إلى التفاصيل التقنية والتنظيمية. فهي تعمل معاً لتعريف صورة كاملة عن النظام المطلوب.

Type of Requirement	Description & Focus	Key Questions Answered	Examples
Business Requirements	High-level organizational goals and the strategic value the system provides.	Why are we building this system? What business value does it deliver?	• Increase customer retention by 20% • Reduce operational costs by 15% • Enter a new market segment
User Requirements	Tasks and capabilities users need to perform with the system, written from their perspective.	What should the user be able to do? How will it help them in their work?	• "As a user, I want to reset my password securely" • "As a manager, I want to generate monthly sales reports"
Functional Requirements	Specific behaviors, functions, and features the system must have to meet user needs.	How will the system accomplish the user tasks? What specific features are needed?	• The system shall validate user passwords against complexity rules • The system shall allow users to filter reports by date range
Non-Functional Requirements	Quality attributes, performance standards, and constraints under which the system operates.	How well must the system perform? What are its quality standards?	• The system must be available 99.9% of the time • The interface must load in under 3 seconds on mobile devices
Domain Requirements	Constraints and rules derived from the specific field or industry where the system will be used.	What specialized rules must the system follow?	• Medical systems must comply with HIPAA regulations • Financial systems must implement specific audit trails

User Requirements vs. System Requirements

Aspect	User Requirements	System Requirements
Definition	High-level statements in natural language about what the system should do	Detailed, structured specifications of the system's functions and constraints
Audience	Customers, end-users, business stakeholders	Developers, testers, system architects, technical team
Format	Natural language, use cases, user stories, diagrams	Technical specifications, structured documents, interface definitions
Level of Detail	WHAT the system should accomplish	HOW the system will implement the requirements
Purpose	Define user needs and expectations	Serve as blueprint for development and testing
Examples	• "Users should be able to search for products by category" • "The system must be easy to navigate" • "Customers should receive order confirmations via email"	• "The search API shall accept category ID as parameter and return JSON response within 500ms" • "User interface shall follow WCAG 2.1 AA accessibility standards" • "Email service shall send confirmation within 2 minutes of order placement"

Key Relationship between User and System Requirements

User Requirements → Define the **WHAT** and **WHY**

Expressed in business language

Focus on user needs and benefits

Measure success by user satisfaction

System Requirements → Define the **HOW**

Expressed in technical language

Focus on implementation details

Measure success by technical compliance

Both are essential for delivering a successful system that meets both user expectations and technical specifications.

User Requirements vs. System Requirements - MHC-PMS Example

Aspect	User Requirements	System Requirements
Definition	High-level business need describing WHAT the system should achieve	Detailed technical specifications describing HOW the system will implement the requirement
Focus	Business value and user needs	Technical implementation and constraints
Example	"The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month."	1.1 On the last working day of each month, generate drug prescription summary 1.2 Generate report for printing after 17:30 1.3 Report per clinic listing drugs, prescriptions, doses, costs 1.4 Separate reports for different dose units 1.5 Restricted access to authorized users only
Characteristics	• Business-oriented • General • Focused on value • Understandable by non-technical users	• Technical • Specific and measurable • Testable • Implementation-focused